

ZERO-DAY NETWORK INTRUSION DETECTION SYSTEM

ZeroDayAI — Complete Project Run Guide & Local Deployment Manual

Document Version: 1.0.0	Operating System: Windows 10/11 (64-bit)
Python Environment: backend/.venv (3.10+)	Node.js Runtime: Node 18+ LTS / npm 9+
Database Engine: MongoDB 6.0+ / 7.0+	Deployment Scope: Local Dev & SOC Testing
Release Date: 2026-09-09	Classification: Operational Technical Standard

[IMPORTANT] Purpose of This Run Guide

This manual provides complete, verified, deterministic instructions for setting up and running the Zero-Day Network Intrusion Detection System from scratch on a fresh machine. Every command listed has been verified against the actual application codebase. All Python operations strictly require **backend/.venv**.

System Architecture Flow

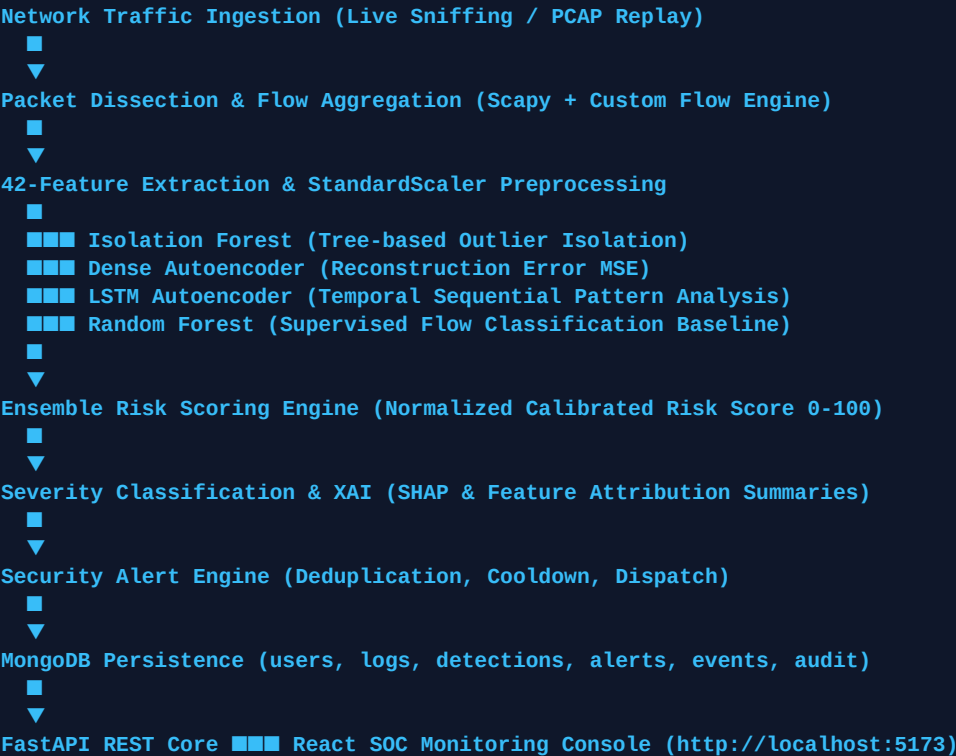


Table of Contents

1. Prerequisites & System Requirements	Page 3
2. Project Directory Structure	Page 3
3. Python Virtual Environment Policy	Page 4
4. Database Setup (MongoDB)	Page 4
5. Environment Variables Configuration	Page 6
6. Backend Setup & Installation	Page 7
7. Database Seeding & RBAC User Management	Page 7
8. Backend Server Startup & URLs	Page 8
9. Frontend Setup, Installation & Startup	Page 8
10. Quick Start Guide (Zero to Running)	Page 10
11. End-to-End Operational Workflow	Page 10
12. Real-Time Monitoring & Live Sniffing	Page 11
13. PCAP Ingestion & Replay Testing	Page 11
14. SOC Dashboard Telemetry & Analytics	Page 12
15. Security Incident Alert Management	Page 12
16. Detection History & Zero-Day Analytics	Page 13
17. AI Ensemble Model Registry	Page 13
18. System Health Diagnostics	Page 14
19. User Management (Admin Console)	Page 14
20. Verification & Test Commands	Page 15
21. Comprehensive Troubleshooting Manual (20 Scenarios)	Page 16
22. Common Windows Administration Commands	Page 17
23. Daily Operations: Startup & Orderly Shutdown	Page 17
24. Security Best Practices & Production Hardening	Page 18
25. Known Limitations & Operational Boundaries	Page 18

1. Prerequisites & System Requirements

Ensure that the host machine satisfies all required runtime dependencies before proceeding with installation. The project has been fully validated on Microsoft Windows 10/11 64-bit environments.

Software Component	Minimum Version	Verified Version	Verification Command
Operating System	Windows 10 64-bit	Windows 11 Pro 64-bit	cmd /c ver
Python	3.10.x	3.14.3 / 3.11.x	python --version
Node.js	18.x LTS	24.14.0	node --version
npm	9.x	10.9.0	npm --version
MongoDB Server	6.0+ / Atlas	7.0.12 / Atlas M0+	mongosh --version
Git	2.30+	2.45.0	git --version

Verification Commands (PowerShell / CMD)

```
python --version
node --version
npm --version
git --version
```

2. Project Directory Structure

The application maintains strict architectural separation between frontend UI code, backend FastAPI services, pre-trained machine learning weights, and operational scripts.

```
ZERO/
├── backend/           # FastAPI Python backend service
│   ├── .env           # Active backend environment variables
│   ├── .env.example   # Backend environment template
│   ├── .venv/         # Mandatory isolated Python virtual environment
│   ├── requirements.txt # Pinned Python package dependencies
│   ├── app/           # Application core, API, ML, and database modules
│   └── tests/         # 210 Pytest automated test suites
├── frontend/         # React 19 + Vite SOC Monitoring Dashboard
│   ├── .env           # Active frontend configuration
│   ├── .env.example   # Frontend environment template
│   ├── package.json    # Node dependencies & test scripts
│   ├── src/           # React components, pages, hooks, and context
│   └── dist/          # Production compiled frontend bundle
├── data/             # Cleaned CSV datasets and PCAP capture files
├── ml_models/        # Serialized AI models, scalers, and explainers
├── scripts/          # Automated setup, startup, and test scripts
└── docs/             # Technical specifications and PDF Run Guide
```

3. Python Virtual Environment Policy

[IMPORTANT] Mandatory Runtime Constraint

ALL Python operations, model loading, user seeding, server startup, and test suites **MUST** be executed using the isolated virtual environment: **backend/.venv**.

DO NOT install Python packages globally. Global packages can lead to dependency conflicts and non-deterministic behavior.

Commands for Virtual Environment Management

```
# 1. Create virtual environment on fresh machine (from project root):
cd backend
python -m venv .venv

# 2. Activate virtual environment on Windows (PowerShell):
.\.venv\Scripts\Activate.ps1

# 2b. Activate virtual environment on Windows (Command Prompt):
.venv\Scripts\activate.bat

# 3. Alternative: Direct Python execution without shell activation:
.\backend\.venv\Scripts\python.exe
```

4. Database Setup (MongoDB)

ZeroDayAI connects asynchronously to MongoDB to persist security events, network flows, AI detections, deduplicated alerts, and user accounts. The target database name is **zero_day_detection** across 6 indexed collections.

6 Core Database Collections

- **users**: RBAC user accounts, salted bcrypt password hashes, and active status.
- **network_logs**: Ingested raw network flow features, IPs, ports, and packet metrics.
- **detection_results**: AI model scores, anomaly flags, severity ratings, and latencies.
- **alerts**: Deduplicated security incidents, triage states, and XAI feature attributions.
- **system_events**: Lifecycle events, monitoring state transitions, and system errors.
- **audit_logs**: Tamper-evident record of logins, user creations, and alert triage.

Starting Local MongoDB (Windows Service)

```
# Check status of MongoDB Windows Service:
Get-Service -Name MongoDB

# Start MongoDB Service (Requires Administrator PowerShell):
Start-Service -Name MongoDB

# Alternative CMD command:
net start MongoDB
```

Verifying MongoDB Connection with Diagnostic Tool

```
# Run direct database verification test:  
.\backend\.venv\Scripts\python.exe scripts/test_mongo_verification.py
```

5. Environment Variables Configuration

Both backend and frontend services rely on environment files. Template configurations are provided in `.env.example`.

Creating Environment Files

```
# Create backend .env (from project root):
Copy-Item backend\.env.example backend\.env

# Create frontend .env:
Copy-Item frontend\.env.example frontend\.env
```

Backend Environment Variables (`backend/.env`)

Variable	Req	Default / Example	Purpose & Description
ENVIRONMENT	Yes	development	Runtime mode: development production testing
API_PORT	Yes	8000	FastAPI HTTP listening port
FRONTEND_URL	Yes	http://localhost:5173	Target client URL for CORS origin validation
CORS_ORIGINS	Yes	http://localhost:5173,http://localhost:3000	Comma-separated list of trusted client origins
MONGODB_URI	Yes	mongodb://localhost:27017	MongoDB connection string (local or Atlas URI)
MONGODB_DATABASE	Yes	zero_day_detection	Target database name for all security collections
JWT_SECRET_KEY	Yes	dev-insecure-secret-key-...	Secret for JWT signature (must change in prod)
MODEL_DIRECTORY	Yes	./ml_models/trained	Path to serialized AI model weights (.joblib, .pt)
DEFAULT_ANOMALY_THRES HOLD	No	0.65	Decision boundary for anomaly classification

Frontend Environment Variables (`frontend/.env`)

- **VITE_API_BASE_URL:** `http://localhost:8000` (FastAPI API gateway)
- **VITE_POLLING_INTERVAL_MS:** `5000` (Telemetry refresh interval in milliseconds)

6. Backend Setup & Installation

Follow these exact sequential steps to configure the backend on a fresh machine:

```
# Step 1: Navigate to backend folder (from project root)
cd backend

# Step 2: Create Python virtual environment
python -m venv .venv

# Step 3: Activate virtual environment (PowerShell)
.\.venv\Scripts\Activate.ps1

# Step 4: Upgrade pip
.\.venv\Scripts\python.exe -m pip install --upgrade pip

# Step 5: Install all backend dependencies
.\.venv\Scripts\pip.exe install -r requirements.txt

# Step 6: Install reportlab for PDF generation
.\.venv\Scripts\pip.exe install reportlab

# Step 7: Create environment file
Copy-Item .env.example .env

# (Alternative: run one-click setup script from project root: scripts\setup_env.bat)
```

7. Database Seeding & RBAC User Management

ZeroDayAI enforces Role-Based Access Control (RBAC). Initial development users for each role tier are provisioned using the seeding script. Passwords are salted and hashed with **bcrypt**.

Role	Default Username	Default Email	Permissions Scope
ADMIN	admin	admin@zeroday.local	Full administrative access: User Management (/users), Alert Triage, System Config.
ANALYST	analyst	analyst@zeroday.local	Security Operations: Alert Triage (Acknowledge, Resolve, Dismiss), Live Monitoring.
VIEWER	viewer	viewer@zeroday.local	Read-only access: SOC Dashboard, Detection History, AI Models, System Health.

Exact User Seeding Command

```
# Seed development users non-interactively (from project root):
.\backend\.venv\Scripts\python.exe scripts/seed_users.py --non-interactive

# To update passwords of existing accounts:
.\backend\.venv\Scripts\python.exe scripts/seed_users.py --update-passwords --non-interactive
```

8. Backend Server Startup & URLs

Start the FastAPI ASGI server with auto-reload enabled using Uvicorn inside the virtual environment.

Exact Backend Startup Command

```
# From project root:
.\backend\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload

# (Alternative batch script: scripts\start_backend.bat)
```

Expected Terminal Startup Output

```
=====
ZERO-DAY NIDS BACKEND
=====
[OK] Configuration loaded
[OK] MongoDB connected successfully
[OK] Database: zero_day_detection
[OK] Connection type: Standalone
[OK] MongoDB ping: 2.1 ms
[OK] ML models available (4/4 loaded)
[OK] Security headers & rate limiting active
[OK] Monitoring subsystem ready
[OK] API ready
=====
Server running on http://0.0.0.0:8000
```

Verified Backend URLs

- **API Base URL:** <http://localhost:8000>
- **Interactive Swagger UI:** <http://localhost:8000/docs>
- **ReDoc API Documentation:** <http://localhost:8000/redoc>
- **OpenAPI JSON Schema:** <http://localhost:8000/openapi.json>
- **Health Status Probe:** <http://localhost:8000/api/v1/health>

9. Frontend Setup, Installation & Startup

The React 19 SOC Operations Center is served via Vite.

Exact Frontend Setup & Startup Commands

```
# Step 1: Open a new terminal and navigate to frontend directory
cd C:\Users\Win\Desktop\ZERO\frontend

# Step 2: Install node dependencies
npm install

# Step 3: Configure environment file
Copy-Item .env.example .env

# Step 4: Start development server
npm run dev

# (Alternative batch script: scripts\start_frontend.bat)
```

Browser Access

Open your web browser and navigate to: <http://localhost:5173>

12. Real-Time Monitoring & Live Sniffing

The **Monitoring** module (/monitoring) provides continuous packet capture, bidirectional flow aggregation, and real-time inference.

- **Select Source:** Choose between *Live Interface Sniffing* and *PCAP File Replay*.
- **Packet Filters:** Apply standard BPF syntax (e.g., tcp or udp, port 80 or port 443).
- **Telemetry Stream:** Observe live counters for Packets Captured, Packet Rate (pkts/sec), Aggregated Flows, Detections, and Triggered Alerts.
- **Graceful Stop:** Click *Stop Monitoring* to flush in-flight flow buffers cleanly to MongoDB.

[NOTE] Windows Packet Capture Driver Requirement

Live network capture on Windows requires the **Npcap** driver installed with administrative privileges. If Npcap is unavailable on your test machine, use **PCAP Ingestion Mode** for offline evaluation.

13. PCAP Ingestion & Replay Testing

PCAP replay allows reproducible evaluation of AI models against standard cyber benchmark traffic traces.

1. Place .pcap or .pcapng files into: C:\Users\Win\Desktop\ZERO\data\sample\
2. Navigate to **Monitoring** (/monitoring) and select **PCAP File Ingestion**.
3. Select target capture file from the dropdown and configure replay speed.
4. Click **Start PCAP Replay** to stream flows into the detection engine.

Generating a Synthetic Test PCAP File

```
# Generate sample network traffic capture file:  
.\backend\.venv\Scripts\python.exe scripts/create_sample_pcap.py
```

14. SOC Dashboard Telemetry & Analytics

The **Dashboard** (/dashboard) provides real-time situational awareness across all detection metrics:

- **Total Detections:** Total volume of network flows evaluated by the AI ensemble.
- **Anomalies Identified:** Number of flows exceeding the calibrated anomaly threshold.
- **Open Security Alerts:** Unresolved incident tickets requiring analyst attention.
- **Critical Alerts:** High-severity incidents with risk score ≥ 75.0 .
- **Average Risk Score:** Global weighted risk gauge across recent traffic.
- **Severity Distribution:** Categorized breakdown of Low, Medium, High, and Critical events.
- **Model Readiness Status:** Individual health indicators for all 4 AI engines.
- **Recent Alerts Stream:** Live table of incoming security alerts with instant triage links.

15. Security Incident Alert Management

The **Alerts** console (/alerts) supports end-to-end incident response workflows.

Severity Tier	Risk Score Range	Alert Generation	XAI Explanation
LOW	0.00 - 24.99	Logged in detection history	Disabled (Noise reduction)
MEDIUM	25.00 - 49.99	Filterable detection record	Optional on demand
HIGH	50.00 - 74.99	Raises Incident Ticket	Enabled (Top 10 Features)
CRITICAL	75.00 - 100.00	Immediate Priority Ticket	Enabled (Full SHAP Breakdown)

Alert Detail Forensics (`/alerts/:alertId`)

- **Network Socket Info:** Source IP/Port, Destination IP/Port, Protocol, and Flow Duration.
- **Multi-Model Consensus:** Individual anomaly scores from Isolation Forest, Autoencoder, LSTM, and Random Forest.
- **XAI Feature Attribution:** Ranked bar chart of network features driving the risk assessment.
- **Triage Actions:** Acknowledge, Resolve, or Dismiss with analyst notes.

16. Detection History & Zero-Day Analytics

The **Detections** module (/detections) logs every flow processed by the ML inference engine.

[NOTE] Scientific Context: Zero-Day Proxy Definition

Evaluation against *unseen attacks* is conducted using held-out attack category splits (e.g., training exclusively on normal traffic and known exploit classes, then testing against held-out classes such as PortScan, DoS, or Botnet).

This serves as a valid **zero-day proxy metric**, but does **NOT** constitute mathematical proof of detecting all real-world zero-day exploits.

17. AI Ensemble Model Registry

The **Models** console (/models) inspects the weights, thresholds, and performance metrics of the 4 ensemble models:

Model Engine	Paradigm	Core Strength / Detection Focus	Weight
Isolation Forest	Unsupervised	Fast isolation of sparse outliers in high-dimensional flow space.	25%
Dense Autoencoder	Unsupervised DL	Reconstruction error (MSE) on non-linear manifold representations.	25%
LSTM Autoencoder	Sequential DL	Recurrent temporal analysis across sliding windows of consecutive packets.	25%
Random Forest	Supervised	High-precision classification baseline for known exploit signatures.	25%

18. System Health Diagnostics

The **System Health** page (/health) performs live probes across all platform dependencies:

- **FastAPI Server:** Online status, process uptime, and request latency.
- **MongoDB Database:** Connection state (CONNECTED), ping latency (<5 ms), server topology (Standalone/ReplicaSet), and pool usage.
- **ML Models Loader:** Status of all 4 pre-trained models in memory (4/4 Loaded).
- **Monitoring Engine:** Status of packet capture workers and buffer capacity.
- **Memory & CPU Telemetry:** Host RAM utilization and execution thread count.

19. User Management (Admin Console)

Accessible exclusively to users with the **ADMIN** role via /users.

- **Account Registry:** View all provisioned users, assigned roles, and creation timestamps.
- **Provision New User:** Create analyst accounts with secure passwords and RBAC role assignments.
- **Active / Inactive Toggle:** Instantly revoke platform access for deactivated users.
- **Audit Trail:** All user creations and role changes are recorded in audit_logs.

20. Verification, Quality Assurance & Test Commands

Run the automated test suites using backend/.venv to verify system integrity before deployment:

1. Backend Pytest Suite (210 Tests)

```
# Run complete backend test suite (from project root):  
.\backend\.venv\Scripts\python.exe -m pytest backend/tests -v  
  
# (Alternative batch script: scripts\test_backend.bat)
```

2. Frontend Vitest Suite (16 Tests)

```
# Run frontend unit and integration tests:  
cd frontend  
npm run test
```

3. Frontend Linting & Production Build

```
# Run Oxlint static analysis:  
cd frontend  
npm run lint  
  
# Build production bundle:  
npm run build
```

4. End-to-End System Integration Test

```
# Run full E2E pipeline verification test:  
.\backend\.venv\Scripts\python.exe scripts/test_phase17_e2e.py
```

21. Comprehensive Troubleshooting Manual

This section documents the 20 most frequent operational issues, their root causes, and exact recovery commands:

Problem / Symptom	Root Cause	Exact Resolution & Recovery Command
1. MongoDB Offline	MongoDB service is stopped.	Start service: Start-Service MongoDB in Admin PowerShell.
2. MongoDB Conn Refused	Port 27017 unreachable or blocked.	Verify mongod.cfg bindIp is 127.0.0.1 and port is 27017.
3. Backend Fails to Start	Global Python invoked instead of venv.	Use: .\backend\.venv\Scripts\python.exe -m uvicorn ...
4. Missing Python Module	Incomplete pip dependency install.	Run: .\backend\.venv\Scripts\pip.exe install -r backend/requirements.txt
5. Wrong IDE Interpreter	VS Code pointing to global Python.	Set interpreter in VS Code to .\backend\.venv\Scripts\python.exe.
6. PowerShell Script Blocked	Execution policy restricted.	Run: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
7. Frontend Won't Start	Missing node_modules directory.	Run: cd frontend && npm install && npm run dev
8. npm Peer Conflict	Conflicting package metadata.	Run: npm install --legacy-peer-deps in frontend directory.
9. CORS Error in Browser	Frontend port missing from CORS_ORIGINS.	Add http://localhost:5173 to CORS_ORIGINS in backend/.env.
10. Login Failure (401)	Database not seeded with users.	Run: .\backend\.venv\Scripts\python.exe scripts/seed_users.py --non-interactive
11. Service Conn Error	FastAPI backend not running on port 8000.	Start backend: .\backend\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --port 8000
12. User Mgmt 403 Forbidden	Logged in as ANALYST or VIEWER.	Log in with ADMIN account (admin / Admin12345!).
13. Monitoring Inactive	Missing Npcap driver on Windows.	Install Npcap driver or switch to PCAP Ingestion Mode.
14. 0 Packets Captured	Wrong capture adapter or restrictive filter.	Set filter to empty or choose correct active network adapter.
15. PCAP Path Rejected	PCAP file outside data/ directory.	Place PCAP file in data/sample/ directory.
16. ML Models 0/4 Loaded	Missing serialized .joblib / .pt files.	Run model training/evaluation: .\backend\.venv\Scripts\python.exe scripts/evaluate_models.py
17. Port 8000/5173 in Use	Orphan process occupying port.	Kill process: Stop-Process -Id (Get-NetTCPConnection -LocalPort 8000).OwningProcess -Force
18. Missing Env Variable	Missing backend/.env file.	Copy template: Copy-Item backend\.env.example backend\.env
19. JWT Token Expired (401)	Session exceeded 60 min lifetime.	Log out and log back in to obtain a fresh access token.
20. 429 Too Many Requests	Rate limit threshold exceeded.	Wait 60s or increase API_RATE_LIMIT_DEFAULT_PER_MINUTE in .env.

22. Common Windows Administration Commands

Quick reference for Windows PowerShell and Command Prompt operations:

Operation	PowerShell Command
Activate venv (PowerShell)	<code>.\backend\.venv\Scripts\Activate.ps1</code>
Activate venv (Command Prompt)	<code>backend\.venv\Scripts\activate.bat</code>
Check Port 8000 (Backend)	<code>Get-NetTCPConnection -LocalPort 8000 -ErrorAction SilentlyContinue</code>
Check Port 5173 (Frontend)	<code>Get-NetTCPConnection -LocalPort 5173 -ErrorAction SilentlyContinue</code>
Kill Process on Port 8000	<code>Stop-Process -Id (Get-NetTCPConnection -LocalPort 8000).OwningProcess -Force</code>
Check MongoDB Service	<code>Get-Service -Name MongoDB</code>
Start MongoDB Service	<code>Start-Service -Name MongoDB</code>
Stop MongoDB Service	<code>Stop-Service -Name MongoDB</code>

23. Daily Operations: Startup & Orderly Shutdown

Daily Startup Sequence (5 Steps)

1. Start MongoDB: `Start-Service -Name MongoDB`
2. Start Backend: `.\backend\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --port 8000 --reload`
3. Start Frontend: `cd frontend && npm run dev`
4. Open Browser: Navigate to `http://localhost:5173`
5. Authenticate: Log in with your assigned RBAC credentials.

Orderly Shutdown Sequence

1. **Stop Live Monitoring:** If active, click *Stop Monitoring* in the UI (/monitoring) to flush flow buffers.
2. **Stop Frontend Server:** Press `Ctrl + C` in the Vite terminal.
3. **Stop Backend Server:** Press `Ctrl + C` in the FastAPI terminal. The lifespan context manager will gracefully close the MongoDB connection pool.
4. **(Optional) Stop MongoDB:** `Stop-Service -Name MongoDB` if host shutdown is desired.

24. Security Best Practices & Production Hardening

- **Rotate JWT Secret Key:** Generate a cryptographically random string (≥ 32 chars) in production.
- **Enforce HTTPS / TLS:** Enable `STRICT_TRANSPORT_SECURITY_ENABLED=True` behind an SSL reverse proxy.
- **Protect Credentials:** Never commit `.env` files to Git repositories.
- **Disable Debug Mode:** Set `DEBUG=False` and `ENVIRONMENT=production` in production.
- **Restrict CORS:** Whitelist only trusted enterprise origins.
- **Isolate Virtual Environment:** Strictly contain all dependencies inside `backend/.venv`.

25. Known Limitations & Operational Boundaries

- **Live Capture Permissions:** Live packet capture on Windows requires Npcap with administrative privileges.
- **Zero-Day Proxy Representation:** Unseen attack detection scores represent an experimental zero-day proxy metric, not a mathematical guarantee of detecting every real-world zero-day exploit.
- **Memory Buffering Bounds:** In-memory flow aggregation is bounded at 10,000 flows to prevent memory exhaustion.
- **Single-Node Architecture:** Default setup is optimized for local evaluation. Enterprise multi-gigabit setups require Kafka message queuing and distributed GPU inference workers.

End of Zero-Day NIDS Run Guide — Developed for Enterprise Cyber Defense & Zero-Day Threat Mitigation.